

Whole-Cell Modelling of *E. coli*
Architecture, Web Interface, and
a Proposed Extension to Human Platelets

Project Report

Steve Haigh

March 2026

Contents

I	The Model	3
1	Introduction	3
2	What the Model Does	3
2.1	What goes in	4
2.2	What comes out	4
2.3	Growth conditions and variants	4
3	An Example: Minimal vs. Rich Media	4
II	Architecture	6
4	Repository Structure	6
5	The Simulation Loop	6
6	Core Abstractions	7
6.1	Processes	7
6.2	States	7
6.3	Listeners and Data I/O	7
6.4	Units	8
III	The Web Interface	8
7	Motivation	8
8	Technology and Architecture	9
8.1	The Inspect Data tab in detail	9
IV	Proposed Extension: Whole-Cell Platelet Model	9
9	Biological Motivation	10
9.1	How platelets differ from <i>E. coli</i>	10

10 Framework Reusability Analysis	10
10.1 Key engine modifications needed	11
11 Data Requirements	11
12 Proposed File Structure	11
13 Implementation Ideas	12
13.1 Phase 1: Understand the engine	12
13.2 Phase 2: Build a minimal platelet stub	12
13.3 Phase 3: Add biology incrementally	12
13.4 Why start with granule release?	13

Part I

The Model

1 Introduction

Most computational models in biology focus on one pathway or one organelle at a time. A *whole-cell model* tries something more ambitious: it simulates every known process inside a living cell, all running simultaneously, so that the emergent behaviour of the whole organism can be studied rather than the behaviour of its parts in isolation.

The first such model, published in 2012 by Karr and colleagues at Stanford University, simulated the complete life cycle of *Mycoplasma genitalium* [5]. The same group — the Covert Lab — has since built a much larger whole-cell model of *Escherichia coli* [10, 1], incorporating roughly 43% of that organism’s characterised genes. This model, called **wcEcoli**, is open-source and publicly available.

The wcEcoli model has been used to study growth-rate control [1], tRNA charging dynamics [3], and antibiotic heterogeneity in colonies [9].

This report provides an overview of the wcEcoli model — what it does, how it works, and what its output looks like — together with a description of the browser-based user interface I have built to make the model easier to use. Finally, and most importantly, it outlines a proposed extension of the modelling framework to simulate *human platelets*.

If you are familiar with tools such as COPASI [4] or Virtual Cell [6], it may help to start with the differences. COPASI and VCell each model a chosen set of reactions: you define the species, write rate laws, and the software integrates the differential equations. wcEcoli does not ask you to write equations. Instead, its authors have already encoded 17 distinct biological processes — metabolism, transcription, translation, DNA replication, cell division, and so on — each using whatever mathematical technique suits it best (flux-balance analysis for metabolism, stochastic kinetics for gene regulation, ordinary differential equations for tRNA charging, and others).

Could a whole-cell model be built in COPASI instead? In principle, COPASI can handle large ODE and stochastic systems, but in practice the approach runs into several difficulties. COPASI represents a model as a single network of reactions and species; there is no built-in mechanism for mixing mathematical formalisms within one simulation — for example, combining flux-balance analysis for metabolism with stochastic kinetics for gene regulation and ODEs for tRNA charging, as wcEcoli does. A user would have to force every process into one formalism or manage multiple COPASI models externally and synchronise their states at each timestep, which would negate much of the tool’s convenience. Performance is also a concern: COPASI’s general-purpose ODE integrators scale poorly beyond a few thousand species, whereas wcEcoli tracks tens of thousands of molecular species and individually resolved macromolecules (each ribosome, each replication fork) at one-second resolution for an entire cell cycle. The wcEcoli framework achieves this by using specialised solvers for each process and a custom integer-partitioning scheme for resource allocation — optimisations that a general-purpose tool cannot easily replicate. COPASI remains an excellent choice for pathway-scale models, but the architectural demands of whole-cell modelling — heterogeneous formalisms, discrete molecule tracking, and tight coupling between dozens of concurrent processes — call for a purpose-built engine.

2 What the Model Does

Every second of simulated time, each of the 17 processes declares what molecules it needs, receives an allocation from a shared pool, and then updates the state of the cell. The result is

a simulation that tracks *individual molecules*: individual ribosomes translating individual mRNAs; individual replication forks progressing along the chromosome. A typical simulation covers one full cell cycle (roughly 25–130 minutes, depending on the growth condition) and produces a detailed trace of every measurable quantity along the way.

2.1 What goes in

The model is parameterised from roughly 80 curated data files drawn from experimental literature and databases such as EcoCyc. A preprocessing step called the *Parameter Calculator* (ParCa) reads these raw data and fits them into a consistent set of initial conditions and rate constants. This step takes about 18 minutes on a modern laptop and needs to be run only once per set of conditions.

2.2 What comes out

The simulation produces time-series data for every tracked quantity at one-second resolution. There are over 80 built-in analysis scripts that generate publication-quality plots from this output. Among the most commonly examined are:

- **Cell mass over time** — dry mass, protein mass, RNA mass, DNA mass, and their fractions.
- **Metabolic fluxes** — central carbon metabolism, amino acid exchange rates, glucose yield.
- **Gene expression** — mRNA counts, protein counts, ribosome and RNA polymerase occupancy.
- **DNA replication** — replication fork position, replisome counts, origin firing.
- **Growth-rate determinants** — tRNA charging levels, ppGpp concentration, ribosomal capacity.

2.3 Growth conditions and variants

The model ships with five named growth conditions:

Table 1: Built-in growth conditions.

Name	Carbon source / supplements	Doubling time
basal	Glucose minimal	~44 min
with_aa	Glucose + all 20 amino acids	~25 min
acetate	Acetate minimal	~136 min
succinate	Succinate minimal	~82 min
no_oxygen	Glucose minimal, anaerobic	~100 min

Beyond fixed conditions, the *timelines* variant system allows media shifts at arbitrary times during a simulation. Twenty-nine pre-defined timelines cover nutrient upshifts, downshifts, carbon-source switches, and transitions to anaerobic growth.

3 An Example: Minimal vs. Rich Media

To illustrate what the model can do, consider a comparison between two growth conditions. In **minimal glucose media**, the cell must synthesise all 20 amino acids and every other building block from glucose and salts. In **rich media** (glucose supplemented with all 20 amino acids), the cell can import amino acids directly and redirect its resources.

This comparison recapitulates one of the best-known results in bacterial physiology: the “growth law” described by Scott et al. [8], which states that the fraction of the cell devoted to ribosomes scales linearly with growth rate. The *wcEcoli* model reproduces this relationship as an emergent property, not as a built-in assumption [1].

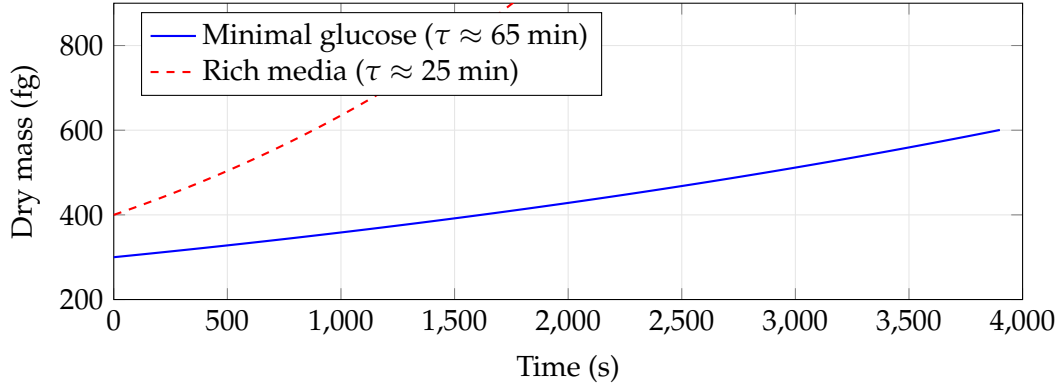


Figure 1: Schematic dry mass trajectories for *E. coli* growing in minimal glucose medium (blue, solid) versus rich medium supplemented with amino acids (red, dashed). The rich-media cell starts larger, grows faster, and divides sooner. Curves are illustrative of the model output; actual traces include stochastic fluctuations.

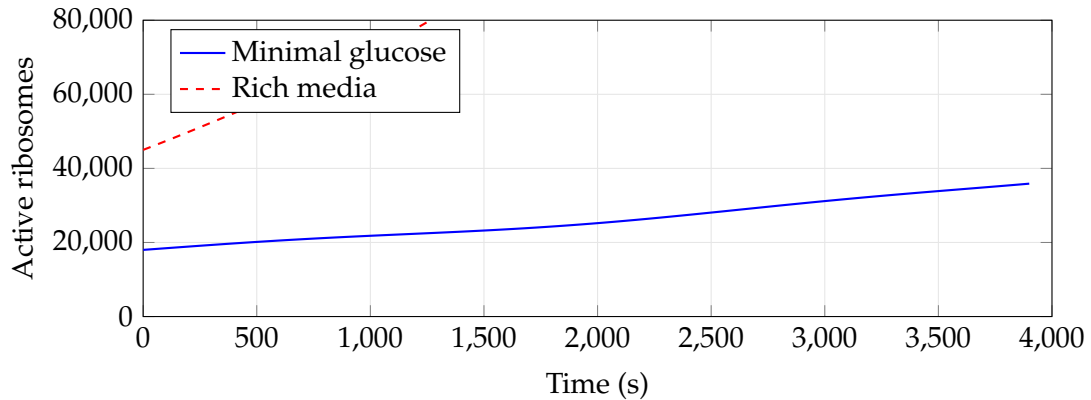


Figure 2: Schematic active ribosome counts. In rich media the cell invests a much larger fraction of its proteome in the translational machinery. This is the “growth law” of Scott et al. [8], reproduced here as an emergent prediction of the whole-cell model.

Table 2 summarises the key differences. The model captures all of these as consequences of its underlying biochemistry, not as separate parameters.

Table 2: Key simulation outputs for two growth conditions.

Metric	Minimal glucose	Rich media
Doubling time	~ 65 min	~ 25 min
Initial dry mass	~ 300 fg	~ 400 fg
Ribosomal protein fraction	~ 15 – 20%	~ 35 – 40%
Biosynthetic enzyme fraction	$\sim 15\%$	~ 3 – 5%
RNA-to-protein ratio	Lower	Higher

The model also supports *nutrient shift* experiments: a simulation can begin in minimal

medium and switch to rich medium (or vice versa) at a specified time. The resulting transient — a burst of rapid growth as the cell reallocates its resources — matches experimental observations of the stringent response [1].

Part II

Architecture

4 Repository Structure

The codebase is written in Python 3.11 with three performance-critical modules compiled via Cython. It is organised into three top-level packages, each with a distinct role:

`wholecell/`

The *generic simulation engine*. Base classes for processes, states, listeners, and views; the simulation loop; binary I/O; a units library; and optional workflow orchestration (FireWorks). This code is organism-agnostic.

`models/ecoli/`

The *E. coli*-specific model. Seventeen process implementations, eighteen listener implementations, ~32 variant definitions, and over 80 analysis plot scripts. Each is a subclass of a framework base class.

`reconstruction/ecoli/`

The Parameter Calculator (ParCa). Reads ~80 flat TSV files of raw experimental data and fits them into a single parameter object (`sim_data`), serialised to disk. The main orchestrator (`fit_sim_data_1.py`) is ~150 KB of Python.

This separation is the key architectural insight: the framework is designed to be reused for other organisms.

5 The Simulation Loop

The simulation is driven by a fixed-timestep loop (default: 1 second). Each tick proceeds through five phases:

1. **Request.** Each process calls `calculateRequest()` to declare how many of each molecule it would like to consume.
2. **Partition.** The `BulkMolecules` state divides the available integer counts among processes according to priority levels (e.g. metabolism at -10 gets first pick; degradation at $+10$ gets last).
3. **Evolve.** Each process calls `evolveState()` to update the cell's state using its allocated molecules.
4. **Merge.** Consumed and produced molecules are reconciled back into the global state.
5. **Listen.** Each listener reads the updated state (read-only) and appends one row to its output table.

Processes run in *dependency-ordered groups*. Within a group, processes share state and run in parallel; between groups, execution is sequential. A typical ordering is: TF unbinding $\rightarrow$ equilibrium $\rightarrow$ TF binding $\rightarrow$ transcription initiation $\rightarrow$ transcript/polypeptide elongation $\rightarrow$... $\rightarrow$ cell division.

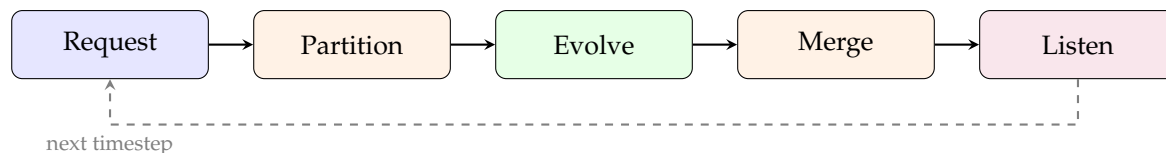


Figure 3: The five phases of each simulation timestep.

6 Core Abstractions

6.1 Processes

Every biological submodel is a subclass of `wholecell.processes.process.Process`. A process must implement:

```

def initialize(self, sim, sim_data):
    """Set up references and constants from sim_data."""
def calculateRequest(self):
    """Declare molecule needs for this timestep."""
def evolveState(self):
    """Update cellular state using allocated molecules."""
  
```

Processes access the cell's state through *views*: `bulkMoleculesView` for integer counts of small molecules and macromolecules; `uniqueMoleculesView` for individually tracked objects (each ribosome, each replication fork); and `environmentView` for external nutrient concentrations.

The 17 *E. coli* processes include metabolism (flux-balance analysis), transcript initiation and elongation, polypeptide initiation and elongation, chromosome replication and structure, transcription factor binding, RNA and protein degradation, complexation, equilibrium, two-component signalling, RNA maturation, and cell division.

6.2 States

Three state containers hold the cell's molecular inventory:

BulkMolecules Integer counts of every molecular species, tagged by compartment ([c] cytoplasm, [p] periplasm, [e] extracellular). The partitioning system divides these counts among processes each timestep.

UniqueMolecules Individually tracked objects with per-instance attributes (e.g. a ribosome's position along an mRNA, a replication fork's chromosomal coordinate).

LocalEnvironment External nutrient concentrations, supporting media-shift timelines.

6.3 Listeners and Data I/O

Listeners are read-only observers that record data each timestep. Each listener writes to a directory of zlib-compressed column files using a custom binary format (`TableWriter/TableReader`). The 18 *E. coli* listeners record mass fractions, metabolic fluxes, ribosome data, replication progress, and more.

Simulation output is stored in a hierarchical directory structure:

```

out/<sim_dir>/
  kb/simData                                # Fitted parameters
  <variant>_<index>/                         # e.g. wildtype_000000/
    <seed>/                                  # e.g. 000000/
      generation_<gen>/
        <daughter>/
  
```

```
simOut/          # Listener data (TableWriter)
plotOut/         # Analysis figures
```

Reading data back requires the TableReader class:

```
from wholecell.io.tablereader import TableReader
reader = TableReader(os.path.join(simout, 'Mass'))
dry_mass = reader.readColumn('dryMass')
```

6.4 Units

Physical quantities carry units throughout the codebase via the Unum library. A value is stripped of its units only at the point of computation:

```
from wholecell.utils import units
conc = 100 * units.mmol / units.L
value = conc.asNumber(units.mol / units.L) # -> 0.1
```

Part III

The Web Interface

7 Motivation

The standard way to use wcEcoli is from the command line. A researcher wishing to run a simulation must type a series of Python commands, wait for them to finish, and then run separate analysis scripts to produce plots. Inspecting specific data columns requires writing short Python scripts using a custom binary data reader.

To lower this barrier, I have built a browser-based user interface (the “webapp”) that wraps the existing model with four interactive tabs:

Configure

A form with quick-start presets (e.g. “Wildtype baseline,” “Nutrient upshift,” “Rich media”) and full control over variant type, number of generations, random seeds, and regulation toggles. Clicking *Run Simulation* submits the job to a background queue.

Run Status

A live-updating table showing each submitted job, its current phase (parameter fitting, simulating, analysing), and elapsed time.

Inspect Data

An interactive chart that lets the user browse any listener and any data column from any completed run. Multiple runs can be overlaid on the same graph for comparison. Transforms (normalisation, log scale) are available as checkboxes.

Explore Plots

A gallery of the pre-generated analysis plots (the same 80+ scripts mentioned earlier), with side-by-side comparison of two runs.

The practical benefit is that a researcher who is not a software developer can configure, run, and inspect simulations entirely from a web browser.

8 Technology and Architecture

The web interface is a **Dash** application (Plotly’s reactive web framework, built on Flask). It runs as a single Python process, serves a browser UI on localhost, and interacts with the simulation code through the same Python API that the command-line scripts use.

Key dependencies: Dash 2.9, Plotly 5.14, Flask (via Dash). The application is containerised via Docker for deployment to Azure Container Instances, with CI/CD via GitHub Actions.

The codebase is organised as follows:

<code>app.py</code>	The application factory. Creates the Dash app, builds all four tab layouts upfront (so every component ID is registered in the initial layout), wires tab-switching via CSS display toggling, and registers each tab’s callbacks.
<code>results.py</code>	Data-access layer. Functions to scan the <code>out/</code> directory tree for simulation runs, variants, cells, listeners, and columns. Also provides the <code>load_column()</code> and <code>load_time()</code> functions that wrap <code>TableReader</code> .
<code>jobs.py</code>	A SQLite-backed job queue. A background thread picks up queued jobs and runs them as subprocesses (either local Python or Docker containers), stepping through phases: <code>queued</code> → <code>parca</code> → <code>simulating</code> → <code>analysing</code> → <code>done/failed</code> .
<code>tabs/</code>	Four tab modules, each exporting a <code>layout()</code> function and a <code>register_callbacks()</code> function: • <code>inspect_data.py</code> — interactive listener data browser • <code>explore.py</code> — analysis plot gallery with side-by-side comparison • <code>configure.py</code> — simulation setup form with presets • <code>runs.py</code> — live job-queue monitor

8.1 The Inspect Data tab in detail

This is the most complex tab. Its callback graph demonstrates how reactive state management works in Dash:

1. **Run dropdown** → populates Listener dropdown (defaults to Mass).
2. **Listener dropdown** → populates Column dropdown (defaults to dryMass for Mass, time for Main).
3. **Column selection + transforms + overlay store** → renders the Plotly figure with primary trace (solid line) and any overlay traces (dashed lines).
4. **Overlay panel**: cascading Run/Listener/Column dropdowns → “Add Trace” button appends to a `dcc.Store` → triggers graph re-render. Each added trace is listed below the panel with a remove button.
5. **Delete button** → `dcc.ConfirmDialog` → on confirm, removes the variant directory from disk; all dropdown options across both Inspect and Explore tabs are refreshed.

Overlay traces use Dash *pattern-matching callbacks* (`{“type”: “remove-overlay-trace”, “index”: ALL}`) to handle a dynamic number of remove buttons without pre-registering each one.

When a run is deleted from the Inspect tab, the callback must also update the Explore tab’s dropdown options and selected values. This is handled by a single callback with outputs spanning both tabs. Shared utility functions live in `results.py` to avoid cross-tab imports.

Part IV

Proposed Extension: Whole-Cell Platelet Model

9 Biological Motivation

Human platelets are anucleate blood cells responsible for haemostasis. They circulate for 7–10 days in a resting state, become activated at sites of vascular injury, and respond by releasing granule contents, changing shape, and aggregating. Despite their clinical importance — they are central to both bleeding disorders and thrombotic disease — no whole-cell model of a platelet exists. Current models focus on individual pathways (calcium signalling, integrin activation, coagulation cascades) but do not integrate them.

9.1 How platelets differ from *E. coli*

The modelling challenges are quite different from those of a bacterium:

- **No genome, no division.** Platelets have no nucleus. There is no DNA replication, no transcription from a genome, and no cell division. The relevant timescale is a *lifespan* (7–10 days), not a cell cycle.
- **Activation, not growth.** The interesting biology is not steady-state growth but a transition: resting → primed → activated → spent. This happens over seconds to minutes.
- **Compartments matter.** Platelets contain dense granules, alpha-granules, an open canalicular system, and mitochondria. Trafficking between these compartments is central to platelet function.
- **Calcium as the central integrator.** Receptor signalling converges on intracellular calcium concentration, which in turn triggers granule release and integrin activation.

10 Framework Reusability Analysis

I have conducted a detailed analysis of every wcEcoli component to assess reusability. The results are summarised in Table 3.

Table 3: Reusability of wcEcoli components for a platelet model.

Component	Verdict	Action
Simulation loop	Fully generic	Subclass
Process base class	Fully generic	Use as-is
View abstractions	Fully generic	Use as-is
InternalState base	Fully generic	Use as-is
Listener base class	Fully generic	Use as-is
TableWriter/Reader	Fully generic	Use as-is
BulkMolecules	Mostly generic	Override one hardcoded ATP reference
UniqueMolecules	Mostly generic	Generalise compartment mass calculation
LocalEnvironment	Mostly generic	Map media timeline to agonist-addition timeline
divide_cell.py	Not needed	Platelets do not divide
models/ecoli/*	Not reusable	Write new platelet code
reconstruction/ecoli/	Not reusable	Write new parameter set
Web UI	Fully reusable	Works with any organism

10.1 Key engine modifications needed

- **Termination condition.** The simulation currently ends when the cell divides. For platelets, it should terminate on a time limit or a biological condition (e.g. complete degranulation). Setting `_raise_on_time_limit = False` in the simulation subclass achieves this.
- **No division function.** Simply never setting `_divideCellFunction` is sufficient; the engine handles this gracefully.
- **Compartment mass.** `UniqueMolecules.calculateMass()` hardcodes compartment 'c'. This needs generalisation for a multi-compartment platelet model (cytosol, dense granules, alpha granules, mitochondria).

11 Data Requirements

A whole-cell model is only as good as the data used to parameterise it. Two published datasets are particularly important for a platelet model:

Burkhart et al. (2012) [2]

The first comprehensive quantitative proteome of human platelets, covering over 4 000 proteins with copy-number estimates. This dataset would serve a role analogous to that of EcoCyc in the *E. coli* model: it provides the initial molecular inventory (what is present and in what quantity) from which to derive initial conditions, enzyme concentrations, and receptor densities. It also enables comparative analysis of structural and functional pathways, which is essential for deciding which processes to include and how to parameterise them.

Purvis et al. (2008) [7]

A molecular signalling model of platelet phosphoinositide and calcium regulation, covering both resting homeostasis and P2Y₁ receptor activation. This paper provides ODE rate constants, calcium store capacities, and IP₃-mediated release kinetics — exactly the quantitative parameters needed to implement the calcium dynamics process. It also serves as a validation target: the platelet model's calcium transients should reproduce the profiles reported in this work.

Additional data sources will be needed as the model grows (e.g. granule secretion kinetics from lumiaggregometry studies, integrin affinity measurements), but these two papers provide the essential foundation for the initial phases described below.

12 Proposed File Structure

```
models/platelet/
  sim/
    simulation.py          # PlateletSimulation(Simulation)
    initial_conditions.py
  processes/
    calcium_dynamics.py    # Store release, influx, buffering
    granule_release.py     # Dense/alpha granule exocytosis
    receptor_signalling.py # GPVI, PAR, P2Y12 -> Ca2+
    protein_decay.py       # Slow mRNA/protein turnover
  listeners/
    mass.py
    calcium.py
    granule_state.py
  analysis/
    single/
      calcium_trace.py
```

```

        granule_secretion.py
reconstruction/platelet/
    simulation_data.py          # SimulationDataPlatelet
    knowledge_base_raw.py
    dataclasses/
        constants.py
        molecule_groups.py

```

13 Implementation Ideas

The following outlines a phased approach to building the platelet model. These are ideas for future work; the scope of the project is limited and time will likely only allow some of these to be tackled. I present them here to show the range of what is possible and to provide a roadmap that could be followed incrementally.

13.1 Phase 1: Understand the engine

Run the *E. coli* model end-to-end. Trace the simulation loop, document the interfaces that a new organism must satisfy, and identify the minimum `sim_data` contract. The key interface points are:

- `submass_name_to_index` and `compartment_abbrev_to_index` (mass accounting)
- `constants.n_avogadro` (unit conversions)
- `internal_state.bulk_molecules.bulk_data` (molecule IDs and masses)
- `internal_state.unique_molecule.unique_molecule_definitions` (can be empty)
- `molecule_groups.*_division` (can be empty lists for a non-dividing cell)

Deliverable: a short architecture document.

13.2 Phase 2: Build a minimal platelet stub

Create `models/platelet/` with one process (e.g. `RestingDecay`), one listener (`Mass`), and a skeleton `SimulationDataPlatelet`. The goal is a simulation that runs, produces output visible in the web UI, and provides a scaffold for incremental development. The initial molecular inventory would draw on copy-number estimates from Burkhart et al. [2].

Estimated scope: ~500 lines of new code.

Deliverable: a runnable platelet stub.

13.3 Phase 3: Add biology incrementally

Each of the following items represents a self-contained extension. They are listed in a logical order of dependency, but any subset could be implemented independently depending on available time:

1. **Calcium dynamics.** Model calcium stores (dense tubular system) with release via IP_3 receptors, plasma-membrane influx (store-operated calcium entry), and cytosolic buffering. Mathematical formalism: ODEs, similar to the tRNA-charging solver already in the *E. coli* model. Rate constants and store capacities would be drawn from Purvis et al. [7], and the resulting calcium transients could be validated against the same paper's reported profiles.
2. **Granule release.** Dense granules and alpha granules as `UniqueMolecule` instances, each with a state attribute (loaded, fusing, released). Exocytosis is triggered stochastically when cytosolic calcium exceeds a threshold, at a rate proportional to $[Ca^{2+}]$. Cargo (ADP, serotonin, fibrinogen, vWF) moves from granule to extracellular compartment.

3. **Receptor signalling.** Coarse-grained models of the three major activation pathways:
 - GPVI (collagen receptor) $\rightarrow$ PLC γ 2 $\rightarrow$ IP₃ $\rightarrow$ Ca²⁺ release
 - PAR-1/4 (thrombin receptors) $\rightarrow$ PLC β $\rightarrow$ IP₃ $\rightarrow$ Ca²⁺ release
 - P2Y₁₂ (ADP receptor) $\rightarrow$ inhibition of adenylyl cyclase $\rightarrow$ amplification of activation
4. **Integrin activation** (optional). α IIB β 3 inside-out signalling, fibrinogen binding, outside-in signalling.
5. **Metabolism** (optional). Mitochondrial ATP production, glycolysis. Lower priority because energy depletion is not the primary driver of acute platelet activation.

Deliverable (if time allows): a version 0.1 model simulating thrombin-induced calcium rise and dense-granule release over five minutes, with secretion kinetics plots.

13.4 Why start with granule release?

Three reasons:

1. Granules are *discrete, countable objects* — a natural fit for the UniqueMolecules state, which already tracks individual ribosomes and replication forks in the *E. coli* model.
2. Granule release has *clear experimental observables*: ATP/ADP secretion profiles, serotonin release kinetics, P-selectin surface expression (measurable by flow cytometry).
3. It *bridges multiple scales*: receptor binding (molecular) $\rightarrow$ calcium signalling (pathway) $\rightarrow$ exocytosis (organelle) $\rightarrow$ extracellular cargo (cell-level output). This is exactly the kind of multi-scale integration that a whole-cell model is designed for.

Summary

The wcEcoli whole-cell model is a mature, open-source simulation of *E. coli* that integrates 17 biological processes and reproduces emergent properties such as growth-rate-dependent ribosome allocation. I have built a web interface that makes it accessible to non-programmers, and I propose to reuse its generic simulation engine to explore the feasibility of building the first whole-cell model of a human platelet, starting with calcium dynamics and granule release. The scope of the proposed work is ambitious; the phased approach allows meaningful progress even if only the earlier stages are completed within the available time.

Use of Artificial Intelligence

This report, the web interface, and supporting code were developed with substantial assistance from AI tools. Specifically:

- **GitHub Copilot** was used throughout the development of the web interface for code completion, boilerplate generation, and exploratory prototyping.
- **Claude Code** (Anthropic's Claude, via the CLI tool) was used for code review, refactoring, architectural analysis of the wcEcoli framework, and generating this report document.

All AI-generated code was reviewed, tested, and modified by the author before inclusion. The biological content and proposed modelling strategy reflect the author's own analysis, informed by the referenced literature, though the prose was drafted with AI assistance and then edited for accuracy and style.

I believe transparency about AI usage is important. These tools significantly accelerated development — particularly for the web interface and for navigating the large wcEcoli codebase — but the scientific judgements, architectural decisions, and biological reasoning are my own.

References

- [1] Travis A. Ahn-Horst et al. “An expanded whole-cell model of *E. coli* links cellular physiology with mechanisms of growth rate control”. In: *npj Systems Biology and Applications* 8.1 (2022), p. 30. DOI: [10.1038/s41540-022-00242-9](https://doi.org/10.1038/s41540-022-00242-9).
- [2] Julia M. Burkhardt et al. “The first comprehensive and quantitative analysis of human platelet protein composition allows the comparative analysis of structural and functional pathways”. In: *Blood* 120.15 (2012), e73–e82. DOI: [10.1182/blood-2012-04-416594](https://doi.org/10.1182/blood-2012-04-416594).
- [3] Heejo Choi and Markus W. Covert. “Whole-cell modeling of *E. coli* confirms that in vitro tRNA aminoacylation measurements are insufficient to support cell growth”. In: *Nucleic Acids Research* 51.12 (2023), pp. 5911–5930. DOI: [10.1093/nar/gkad435](https://doi.org/10.1093/nar/gkad435).
- [4] Stefan Hoops et al. “COPASI — a COMplex PATHway SIMulator”. In: *Bioinformatics* 22.24 (2006), pp. 3067–3074. DOI: [10.1093/bioinformatics/btl485](https://doi.org/10.1093/bioinformatics/btl485).
- [5] Jonathan R. Karr et al. “A whole-cell computational model predicts phenotype from genotype”. In: *Cell* 150.2 (2012), pp. 389–401. DOI: [10.1016/j.cell.2012.05.044](https://doi.org/10.1016/j.cell.2012.05.044).
- [6] Ion I. Moraru et al. “Virtual Cell modelling and simulation software environment”. In: *IET Systems Biology* 2.5 (2008), pp. 352–362. DOI: [10.1049/iet-syb:20080102](https://doi.org/10.1049/iet-syb:20080102).
- [7] J. E. Purvis et al. “A molecular signaling model of platelet phosphoinositide and calcium regulation during homeostasis and P2Y1 activation”. In: *Blood* 112.10 (2008), pp. 4069–4079. DOI: [10.1182/blood-2008-05-157883](https://doi.org/10.1182/blood-2008-05-157883).
- [8] Matthew Scott et al. “Interdependence of cell growth and gene expression: origins and consequences”. In: *Science* 330.6007 (2010), pp. 1099–1102. DOI: [10.1126/science.1192588](https://doi.org/10.1126/science.1192588).
- [9] Christopher J. Skalnik et al. “Whole-cell modeling of *E. coli* colonies enables quantification of single-cell heterogeneity in antibiotic responses”. In: *PLOS Computational Biology* 19.6 (2023), e1011232. DOI: [10.1371/journal.pcbi.1011232](https://doi.org/10.1371/journal.pcbi.1011232).
- [10] Gwanggyu Sun, Travis A. Ahn-Horst, and Markus W. Covert. “The *E. coli* whole-cell modeling project”. In: *EcoSal Plus* 9.2 (2021), eESP-0001–2020. DOI: [10.1128/ecosalplus.ESP-0001-2020](https://doi.org/10.1128/ecosalplus.ESP-0001-2020).